

Réseaux de neurones : présentation du fascicule

Ce dernier fascicule présente les modèles qui ont transformé la discipline depuis 2012. Ils reposent sur les mêmes principes que le fascicule 3 : ajuster les paramètres d'une fonction pour minimiser une perte.

Ce qui les distingue est la manière dont la fonction est construite : par composition de transformations simples, empilées en couches, plutôt que par une forme fixée à l'avance.

Cette souplesse explique à la fois leur succès sur des données non structurées et la difficulté de leur interprétation. Il n'existe pas de règle lisible associée à un réseau entraîné.

Les questions d'évaluation restent celles du fascicule 4. Aucune architecture ne dispense d'un protocole expérimental rigoureux.

Du modèle linéaire au neurone

Un neurone artificiel calcule une somme pondérée de ses entrées, puis applique une fonction non linéaire au résultat. C'est une généralisation immédiate de la régression linéaire.

La fonction appliquée à la sortie s'appelle fonction d'activation. Sans elle, l'empilement de couches se réduirait à une seule transformation linéaire, quelle que soit la profondeur du réseau.

Le terme neurone est un héritage historique. Il ne suggère aucune propriété biologique : c'est un opérateur mathématique, dont les paramètres sont ajustés par optimisation.

La terminologie reste trompeuse dans un autre sens : un réseau de neurones n'apprend ni ne comprend. Il ajuste des coefficients jusqu'à obtenir un comportement conforme aux exemples.

Le perceptron

Le perceptron, proposé par Rosenblatt en 1958, est le premier modèle de ce type à avoir été implémenté. Il produit une sortie binaire à partir d'une combinaison linéaire des entrées.

Sa règle d'apprentissage est simple : lorsqu'un exemple est mal classé, on modifie les poids dans la direction qui corrige cette erreur.

Il a été démontré qu'il trouve une solution en temps fini lorsque les classes sont linéairement séparables. Sa convergence est donc garantie, mais seulement dans ce cas.

L'engouement initial a été suivi d'une désillusion rapide, lorsque ses limites ont été rendues publiques. Cet épisode est l'un des hivers de l'intelligence artificielle évoqués dans le fascicule 0.

La limite du perceptron simple

Un perceptron ne peut pas représenter la fonction ou-exclusif, dont les exemples positifs et négatifs ne sont pas séparables par une droite.

Minsky et Papert ont publié en 1969 une analyse systématique de ces limites. Leur conclusion, souvent résumée de façon abusive, a freiné les recherches sur les réseaux pendant plus d'une décennie.

L'important dans ce résultat n'est pas la limitation elle-même, mais ce qu'elle révèle : certaines relations exigent une représentation intermédiaire qui ne s'écrit pas directement en fonction des entrées.

C'est précisément ce que fournit l'empilement de couches. Les neurones intermédiaires calculent des combinaisons utiles, que la couche de sortie peut ensuite combiner linéairement.

Réseaux à plusieurs couches

Un réseau à une couche cachée comporte trois étages : les entrées, une transformation non linéaire vers un espace intermédiaire, puis une sortie.

Sous des hypothèses larges, un tel réseau peut approximer n'importe quelle fonction continue sur un domaine borné, à condition d'avoir suffisamment de neurones cachés.

Ce résultat d'approximation universelle est une garantie d'existence, pas de faisabilité : il ne dit rien sur le nombre de neurones nécessaires, ni sur la possibilité effective de les ajuster par apprentissage.

En pratique, le choix se porte sur des architectures profondes plutôt que très larges, parce qu'elles réutilisent mieux leurs paramètres.

Fonctions d'activation

La fonction logistique, ou sigmoïde, écrase la sortie dans l'intervalle entre zéro et un. Elle a été longtemps la référence, avant que ses inconvénients numériques ne soient identifiés.

La tangente hyperbolique est son équivalent centré sur zéro. Ce centrage accélère la convergence dans la plupart des cas.

La fonction rectifiée, nulle pour les valeurs négatives et égale à la valeur pour les positives, est aujourd'hui l'option par défaut dans les couches cachées. Son gradient est constant sur la partie positive, ce qui limite la disparition du signal.

Des variantes lisses ou paramétrées existent, mais leurs gains sont marginaux sur la plupart des problèmes.

Propagation avant

La propagation avant consiste à calculer la sortie du réseau pour une entrée donnée, couche après couche. Chaque couche applique sa transformation puis sa fonction d'activation.

Cette étape est un simple calcul de composition de fonctions. Elle est entièrement déterministe pour des paramètres fixés.

Sur des lots de plusieurs exemples, les calculs sont exprimés par des opérations matricielles, ce qui permet d'exploiter efficacement le matériel disponible.

La propagation avant est aussi l'opération exécutée en production : une fois le modèle entraîné, prédire revient exactement à ce calcul.

Sortie et perte

La forme de la couche de sortie dépend de la tâche. Pour une régression, on utilise une sortie linéaire. Pour une classification binaire, une probabilité. Pour une classification à plusieurs classes, une distribution normalisée.

La perte associée est choisie en conséquence : erreur quadratique pour la régression, perte logistique pour le binaire, entropie croisée pour le multi-classe.

Ces choix ne sont pas arbitraires. Ils correspondent à maximiser la vraisemblance des données sous une hypothèse de distribution donnée.

La fonction utilisée pour transformer les scores en distribution, appelée softmax, produit des valeurs positives de somme égale à un. Elle est souvent combinée à l'entropie croisée, pour des raisons de stabilité numérique.

Rétropropagation du gradient

La rétropropagation calcule le gradient de la perte par rapport à chaque paramètre. Elle applique la règle de dérivation des fonctions composées, en parcourant le réseau de la sortie vers l'entrée.

Pour une composition de fonctions, la dérivée est le produit des dérivées intermédiaires. Le calcul du gradient se fait donc en multipliant des termes le long du chemin, d'où le risque de disparition ou d'explosion du signal.

L'algorithme réutilise les résultats des couches suivantes, ce qui évite de recalculer chaque dérivée partielle indépendamment. Son coût est comparable à celui d'une propagation avant.

Sa redécouverte et sa popularisation au milieu des années 1980 ont relancé l'entraînement des réseaux multicouches, dont la règle d'apprentissage était jusque-là inconnue.

Graphes de calcul et différenciation automatique

Un graphe de calcul représente chaque opération élémentaire comme un noeud, et chaque dépendance entre valeurs comme une arête.

La différenciation automatique applique la règle de la chaîne sur ce graphe. Elle diffère de la différenciation symbolique, qui peut produire des expressions énormes, et de la différenciation numérique, qui est imprécise.

Les bibliothèques modernes construisent ce graphe à l'exécution, à partir des opérations réellement effectuées. Cette approche simplifie l'écriture de modèles complexes.

Elle explique aussi pourquoi les fuites de mémoire sont fréquentes : chaque valeur intermédiaire reste référencée tant que le graphe n'est pas libéré.

Descente de gradient stochastique

Calculer le gradient sur la totalité du jeu d'entraînement à chaque étape est coûteux et inutile lorsque les données sont redondantes. La descente stochastique estime le gradient sur un petit lot tiré au hasard.

Cet estimateur est bruité, mais non biaisé. Le bruit permet de traverser des régions plates et de sortir de certains points selles.

Le taux d'apprentissage reste le paramètre le plus critique. Sa valeur se choisit expérimentalement, souvent par une recherche sur une échelle logarithmique.

Des politiques de décroissance font varier ce taux au fil de l'entraînement : grand au début pour avancer vite, petit ensuite pour affiner.

Variantes modernes de l'optimisation

La méthode du moment accumule une moyenne des gradients passés, ce qui amortit les oscillations dans les directions où le gradient change de signe.

Les méthodes à taux adaptatif divisent le pas par une estimation de l'amplitude des gradients récents, variable par variable. Elles sont robustes et demandent peu de réglage.

L'algorithme Adam, proposé en 2014, combine ces deux idées. C'est l'option par défaut dans la majorité des implémentations actuelles.

Ces méthodes n'offrent pas de garantie de convergence supérieure. Sur certaines tâches, une descente stochastique bien réglée les dépasse, au prix d'un réglage plus fin.

Taille des lots et boucles d'entrainement

La taille du lot contrôle l'équilibre entre le cout d'un pas et la qualité de l'estimation du gradient. Des lots plus grands donnent un gradient plus fidèle, mais moins de mises à jour par époque.

L'époque désigne une passe complète sur le jeu d'entrainement. Le nombre d'époques se détermine par arrêt précoce, en observant le score de validation.

L'ordre de présentation des exemples influence le résultat. Mélanger les données à chaque époque est la pratique standard, et son absence est une cause fréquente de résultats inexplicables.

Enfin, la reproduction d'un entrainement suppose de fixer la graine du générateur aléatoire. Sans cette précaution, deux exécutions du même code donnent des résultats différents.

Initialisation des paramètres

Initialiser tous les poids à zéro empêche l'apprentissage : tous les neurones d'une même couche calculent la même chose et reçoivent le même gradient.

L'initialisation aléatoire doit respecter l'échelle des activations. Une initialisation trop grande fait diverger le signal, trop petite l'éteint au fil des couches.

Les schémas usuels tirent les poids dans une distribution dont la variance dépend du nombre d'entrées et de sorties de la couche. Ce calibrage maintient l'ordre de grandeur des activations en profondeur.

Ces règles sont intégrées aux bibliothèques. Les modifier sans raison est rarement utile, mais comprendre leur rôle aide à diagnostiquer les entraînements qui ne démarrent pas.

Régularisation des réseaux

La régularisation par pénalité des poids s'applique comme dans le fascicule 4. Elle limite l'amplitude des paramètres et réduit la variance.

L'abandon aléatoire, ou dropout, désactive une fraction des neurones à chaque pas d'entraînement. Le réseau ne peut plus compter sur un neurone particulier, ce qui l'oblige à répartir l'information.

L'arrêt précoce reste le régularisateur le plus simple et le plus efficace : on arrête l'entraînement lorsque le score de validation cesse de progresser.

L'augmentation des données est également une forme de régularisation, particulièrement adaptée aux images, où des transformations géométriques produisent des exemples plausibles supplémentaires.

Normalisation des activations

La normalisation par lots ramène les activations de chaque couche à une moyenne et un écart-type contrôlés, calculés sur le lot courant.

Elle accélère l'entraînement et permet des taux d'apprentissage plus élevés. Elle atténue aussi la sensibilité à l'initialisation.

Son inconvénient apparaît lorsque la taille des lots est petite : les statistiques estimées deviennent bruitées. Des variantes calculent ces statistiques sur d'autres axes.

À l'inférence, les statistiques ne sont plus calculées sur le lot courant mais estimées pendant l'entraînement. Cette différence de comportement entre entraînement et inférence est une source classique de bogues discrets.

Diagnostiquer un entraînement

Un signal qui disparaît couche après couche empêche les premières couches d'apprendre. C'est le cas avec des fonctions d'activation saturantes et une initialisation inadaptée.

Un signal qui explose produit des valeurs non finies et interrompt l'entraînement. La coupure du gradient, qui borne sa norme, est le remède standard.

Une perte qui ne diminue pas peut venir d'un taux d'apprentissage trop grand, d'une erreur dans les données, ou d'une couche de sortie mal adaptée à la tâche.

Suivre la perte d'entraînement et la perte de validation, ainsi que la norme des gradients, permet de distinguer ces situations. Journaliser des mesures adaptées coûte moins de temps qu'explorer les hyperparamètres au hasard.

Réseaux convolutifs

Un réseau convolutif applique des filtres de petite taille sur des voisinages de l'entrée. Chaque filtre partage ses paramètres sur toute l'image.

Ce partage réduit massivement le nombre de paramètres et introduit une invariance par translation : un motif est détecté au même endroit quel que soit sa position dans l'image.

L'empilement de couches de convolution, entrecoupées d'opérations de sous-échantillonnage, produit des représentations de plus en plus abstraites, des contours jusqu'aux parties d'objets.

Ce principe découle d'une connaissance du domaine, à savoir la structure locale des images. C'est pourquoi il reste supérieur, sur ce type de données, à des architectures entièrement génériques.

Les architectures qui ont marqué la période

Les premiers réseaux convolutifs appliqués à la reconnaissance de chiffres manuscrits datent de la fin des années 1980. Ils contenaient déjà les éléments essentiels des architectures actuelles.

En 2012, un réseau convolutif profond remporte une compétition de reconnaissance d'images avec une avance considérable sur les méthodes classiques. Cet épisode marque le début de la vague actuelle.

Les années suivantes voient l'apparition d'architectures toujours plus profondes, rendues entraînables par des connexions résiduelles qui facilitent la circulation du gradient.

Ces architectures reposent sur une idée simple : permettre au signal de traverser de nombreuses couches sans se dégrader.

Séquences et réseaux récurrents

Un réseau récurrent traite une séquence en conservant un état interne mis à jour à chaque élément. La même transformation est appliquée à chaque pas.

Cette structure convient aux textes, aux séries temporelles et aux signaux, dont la longueur varie et dont l'ordre porte l'information.

Elle souffre d'une difficulté d'optimisation : le gradient est multiplié de nombreuses fois le long de la séquence, ce qui le fait disparaître ou exploser.

Des variantes à portes, apparues dans les années 1990, atténuent ce problème en apprenant ce qu'il faut conserver et ce qu'il faut oublier. Elles ont longtemps été l'état de l'art sur les séquences.

Mécanismes d'attention

L'attention pondère les éléments d'une entrée en fonction de leur pertinence pour l'élément courant. Chaque position peut donc consulter directement toutes les autres.

Cette opération supprime la contrainte de séquentialité : contrairement à un réseau récurrent, tous les calculs peuvent être menés en parallèle.

L'architecture fondée uniquement sur l'attention, publiée en 2017, a transformé le traitement du langage, puis progressivement d'autres domaines.

Son coût croît quadratiquement avec la longueur de la séquence, ce qui a motivé de nombreux travaux sur des variantes plus économes.

Pré-entraînement et adaptation

Le pré-entraînement consiste à entraîner un modèle sur une grande quantité de données non étiquetées, puis à l'adapter à une tâche précise avec peu d'exemples annotés.

Cette stratégie s'est généralisée au point de devenir la norme. Elle déplace l'effort : l'essentiel du coût se situe dans la première étape.

Les représentations apprises sont réutilisables. Un modèle pré-entraîné sur du texte fournit des vecteurs utiles pour la recherche d'information, la classification ou la similarité sémantique.

L'adaptation par réglage complet des paramètres est coûteuse. Des méthodes plus économiques n'ajustent qu'un petit nombre de paramètres supplémentaires, ou conditionnent le modèle par des exemples fournis en entrée.

Représentations et plongements

Un plongement associe à un objet discret, mot, image ou utilisateur, un vecteur de nombres réels. Les objets semblables se retrouvent proches dans cet espace.

Ces représentations sont apprises conjointement avec la tâche, sans supervision directe sur leur forme. Leur structure émerge des régularités du jeu de données.

Elles servent de base à de nombreuses applications : recherche sémantique, recommandation, déduplication, détection de similarité entre documents.

Leur interprétation reste limitée. On peut mesurer des proximités et des directions privilégiées, mais attribuer un sens précis à une coordonnée n'est généralement pas possible.

Mise à l'échelle et loi d'échelle

Depuis 2020, l'observation empirique la plus discutée est la suivante : la performance des grands modèles s'améliore de façon prévisible lorsqu'on augmente simultanément le nombre de paramètres et le volume de données.

Ces régularités ont orienté des investissements considérables vers des modèles toujours plus grands, dans l'hypothèse que la tendance se poursuive.

Elles ont aussi montré leurs limites : au-delà d'un certain point, l'ajout de paramètres sans données supplémentaires n'apporte pas d'amélioration proportionnelle.

Rien dans ces observations ne remplace les notions des fascicules précédents. Un grand modèle mal évalué reste un modèle mal évalué.

Cout de calcul et d'énergie

L'entraînement des grands modèles mobilise des moyens matériels considérables, souvent comparables à la consommation annuelle de plusieurs centaines de foyers.

L'inférence, c'est-à-dire l'utilisation du modèle, domine largement le cout total lorsque le service est très sollicité. Un modèle légèrement moins précis mais trois fois plus rapide peut être le meilleur choix industriel.

Ces contraintes ont relancé l'intérêt pour la compression, la quantification et les architectures économes. Ce sont des questions techniques autant qu'économiques.

Elles invitent aussi à se demander si la tâche justifie le moyen. Beaucoup de problèmes traités par un grand modèle sont résolus par une méthode simple, à une fraction du cout.

Interprétabilité et limites

Un réseau entraîné n'expose pas de règle lisible. Les techniques d'interprétation produisent des indications locales, sous forme de zones importantes ou d'attributions, sans reconstituer le raisonnement.

Ces indications sont utiles pour détecter des comportements anormaux, par exemple lorsqu'un modèle s'appuie sur un élément non pertinent de l'image.

Elles ne constituent pas une justification. Un modèle peut produire une attribution convaincante tout en étant faux, et l'inverse.

Enfin, ces modèles produisent des sorties plausibles sans aucune garantie de vérité. La vérification des réponses reste, dans la plupart des usages, une obligation de l'utilisateur ou du système qui l'encadre.

Ce qu'il faut retenir de ce fascicule

Un réseau de neurones est une fonction composée, ajustée par descente de gradient. Ses paramètres n'ont pas d'interprétation individuelle.

La rétropropagation rend le calcul du gradient tractable, à condition que le signal ne disparaisse ni n'explose au fil des couches. C'est le principal point de vigilance en pratique.

Les bonnes architectures encodent une connaissance du domaine : partage de paramètres pour les images, attention pour les séquences. Le choix architectural n'est jamais neutre.

Enfin, le pré-entraînement a déplacé l'effort vers l'amont, mais il n'exempte ni d'un protocole d'évaluation rigoureux, ni d'une analyse du coût d'exploitation.

Exercices du fascicule 5

Exercice 1. Implémenter un perceptron simple et montrer sur un exemple que la fonction ou-exclusif n'est pas apprenable sans couche cachée.

Exercice 2. Calculer à la main les gradients d'un réseau à une couche cachée de deux neurones, pour un exemple unique et une perte quadratique.

Exercice 3. Comparer deux entraînements identiques à la graine aléatoire près, et mesurer l'écart de performance sur le jeu de validation.

Exercice 4. Sur une tâche de classification d'images fournie, évaluer l'effet de l'augmentation des données et de l'abandon aléatoire, séparément puis ensemble.